

Move Validation Code

```
import java.util.ArrayList;
import java.util.List;

public class Board {
    private Piece[][] grid = new Piece[8][8];
    private boolean whiteToMove = true;
    private int[] enPassantTarget = null;
    public int[] lastMoveFrom = null;
    public int[] lastMoveTo = null;
    public Board() {
        this.setupBoard();
    }

    public void setupBoard() {
        this.grid = new Piece[8][8];
        this.whiteToMove = true;
        this.enPassantTarget = null;
        this.lastMoveFrom = null;
        this.lastMoveTo = null;

        for(int var1 = 0; var1 < 8; ++var1) {
            this.grid[6][var1] = new Pawn(true, 6, var1);
            this.grid[1][var1] = new Pawn(false, 1, var1);
        }

        this.grid[7][0] = new Rook(true, 7, 0);
        this.grid[7][7] = new Rook(true, 7, 7);
        this.grid[0][0] = new Rook(false, 0, 0);
        this.grid[0][7] = new Rook(false, 0, 7);
        this.grid[7][1] = new Knight(true, 7, 1);
        this.grid[7][6] = new Knight(true, 7, 6);
        this.grid[0][1] = new Knight(false, 0, 1);
        this.grid[0][6] = new Knight(false, 0, 6);
        this.grid[7][2] = new Bishop(true, 7, 2);
        this.grid[7][5] = new Bishop(true, 7, 5);
        this.grid[0][2] = new Bishop(false, 0, 2);
        this.grid[0][5] = new Bishop(false, 0, 5);
        this.grid[7][3] = new Queen(true, 7, 3);
        this.grid[0][3] = new Queen(false, 0, 3);
        this.grid[7][4] = new King(true, 7, 4);
        this.grid[0][4] = new King(false, 0, 4);
    }

    public Piece getPiece(int var1, int var2) {
```

```

    return var1 >= 0 && var1 <= 7 && var2 >= 0 && var2 <= 7 ? this.grid[var1][var2] : null;
}

```

```

public boolean isWhiteToMove() {
    return this.whiteToMove;
}

```

```

public int[] getEnPassantTarget() {
    return this.enPassantTarget;
}

```

```

public boolean isSquareAttacked(int var1, int var2, boolean var3) {
    for(int var4 = 0; var4 < 8; ++var4) {
        for(int var5 = 0; var5 < 8; ++var5) {
            Piece var6 = this.grid[var4][var5];
            if (var6 != null && var6.isWhite() == var3) {
                for(int[] var8 : var6.getAttackSquares(this)) {
                    if (var8[0] == var1 && var8[1] == var2) {
                        return true;
                    }
                }
            }
        }
    }
}

return false;
}

```

```

public int[] findKing(boolean var1) {
    for(int var2 = 0; var2 < 8; ++var2) {
        for(int var3 = 0; var3 < 8; ++var3) {
            if (this.grid[var2][var3] instanceof King && this.grid[var2][var3].isWhite() == var1) {
                return new int[]{var2, var3};
            }
        }
    }
}

return null;
}

```

```

public boolean isKing(boolean var1) {
    int[] var2 = this.findKing(var1);
    return var2 == null ? false : this.isSquareAttacked(var2[0], var2[1], !var1);
}

```

```

public List<int[]> getLegalMoves(Piece var1) {

```

```

ArrayList var2 = new ArrayList();
if (var1 == null) {
    return var2;
} else {
    for(int[] var4 : var1.getPseudoLegalMoves(this)) {
        Board var5 = this.copy();
        Piece var6 = var5.getPiece(var1.getRow(), var1.getCol());
        var5.executeRawMove(var6, var4[0], var4[1], "Queen");
        if (!var5.isKingI(var1.isWhite())) {
            var2.add(var4);
        }
    }

    return var2;
}
}

public boolean hasAnyLegalMove(boolean var1) {
    for(int var2 = 0; var2 < 8; ++var2) {
        for(int var3 = 0; var3 < 8; ++var3) {
            Piece var4 = this.grid[var2][var3];
            if (var4 != null && var4.isWhite() == var1 && !this.getLegalMoves(var4).isEmpty()) {
                return true;
            }
        }
    }

    return false;
}

private MoveInfo executeRawMove(Piece var1, int var2, int var3, String var4) {
    int var5 = var1.getRow();
    int var6 = var1.getCol();
    Piece var7 = this.grid[var2][var3];
    boolean var8 = false;
    if (var1 instanceof Pawn && var7 == null && var6 != var3) {
        var7 = this.grid[var5][var3];
        this.grid[var5][var3] = null;
        var8 = true;
    }

    this.grid[var5][var6] = null;
    this.grid[var2][var3] = var1;
    var1.setPosition(var2, var3);
    var1.setMoved(true);
    boolean var9 = false;
    if (var1 instanceof King && Math.abs(var3 - var6) == 2) {

```

```

var9 = true;
int var10 = var3 > var6 ? 7 : 0;
int var11 = var3 > var6 ? var3 - 1 : var3 + 1;
Piece var12 = this.grid[var2][var10];
this.grid[var2][var10] = null;
this.grid[var2][var11] = var12;
if (var12 != null) {
    var12.setPosition(var2, var11);
    var12.setMoved(true);
}
}

boolean var13 = false;
if (var1 instanceof Pawn) {
    int var14 = var1.isWhite() ? 0 : 7;
    if (var2 == var14) {
        var13 = true;
        Piece var15 = this.createPiece(var4, var1.isWhite(), var2, var3);
        var15.setMoved(true);
        this.grid[var2][var3] = var15;
    }
}

this.enPassantTarget = null;
if (var1 instanceof Pawn && Math.abs(var2 - var5) == 2) {
    this.enPassantTarget = new int[] {(var5 + var2) / 2, var6};
}

return new MoveInfo(var7, var9, var8, var13);
}

private Piece createPiece(String var1, boolean var2, int var3, int var4) {
    switch (var1) {
        case "Rook" -> {
            return new Rook(var2, var3, var4);
        }
        case "Bishop" -> {
            return new Bishop(var2, var3, var4);
        }
        case "Knight" -> {
            return new Knight(var2, var3, var4);
        }
        default -> {
            return new Queen(var2, var3, var4);
        }
    }
}

```

```

}

public MoveInfo makeMove(int var1, int var2, int var3, int var4, PromotionChooser var5) {
    Piece var6 = this.getPiece(var1, var2);
    if (var6 != null && var6.isWhite() == this.whiteToMove) {
        boolean var7 = false;

        for(int[] var9 : this.getLegalMoves(var6)) {
            if (var9[0] == var3 && var9[1] == var4) {
                var7 = true;
                break;
            }
        }

        if (!var7) {
            return null;
        } else {
            String var10 = "Queen";
            if (var6 instanceof Pawn) {
                int var11 = var6.isWhite() ? 0 : 7;
                if (var3 == var11 && var5 != null) {
                    var10 = var5.choosePromotion(var6.isWhite());
                }
            }
        }

        MoveInfo var12 = this.executeRawMove(var6, var3, var4, var10);
        if (var12.captured != null) {
            if (var6.isWhite()) {
                this.capturedByWhite.add(var12.captured);
            } else {
                this.capturedByBlack.add(var12.captured);
            }
        }

        this.lastMoveFrom = new int[]{var1, var2};
        this.lastMoveTo = new int[]{var3, var4};
        this.whiteToMove = !this.whiteToMove;
        return var12;
    }
    else {
        return null;
    }
}

public Board copy() {
    Board var1 = new Board();

```

```

var1.grid = new Piece[8][8];

for(int var2 = 0; var2 < 8; ++var2) {
    for(int var3 = 0; var3 < 8; ++var3) {
        var1.grid[var2][var3] = this.grid[var2][var3] == null ? null : this.grid[var2][var3].copy();
    }
}

var1.whiteToMove = this.whiteToMove;
var1.enPassantTarget = this.enPassantTarget == null ? null : new
int[] {this.enPassantTarget[0], this.enPassantTarget[1]};
return var1;
}

public static class MoveInfo {
    public final boolean castle;
    public final boolean enPassant;
    public final boolean promotion;

    MoveInfo(Piece var1, boolean var2, boolean var3, boolean var4) {
        this.castle = var2;
        this.enPassant = var3;
        this.promotion = var4;
    }
}

public interface PromotionChooser {
    String choosePromotion(boolean var1);
}
}

```